CLEAN CODE

TIAGO HENRIQUE ANGIOLETTI

ELEGANT

Simple and Direct

LITERATE

Someone who cares

SMALL, EXPRESSIVE, AND SIMPLE

What you expected

THE COST OF BAD CODE

A popular company lost its market due to customer complaints about bugs and extreme slowness. Developers admitted the code was a mess because they rushed the release, planning to clean it later. However, as functionality increased, maintenance became nearly impossible, ultimately forcing the company out of the market.

LEBLANC'S LAW

LATER IS THE SAME AS NEVER.

WHY BAD CODE HAPPENS

Developers complain about SLAs.

Developers complain about customer pressure.

But these are not excuses! Good code should be a professional standard.

THE BROKEN WINDOW THEORY

Bad code accumulates if not fixed early.

A messy codebase leads to a culture of neglect.

YAGNI

YOU AIN'T GONNA NEED IT – Don't add unnecessary features.

SCOUT RULE

ALWAYS LEAVE THE CODE CLEANER THAN YOU FOUND IT.

NOMENCLATURE

NAMES THAT REVEAL PURPOSE

BAD:

What is the content of the list?

GOOD:

Use meaningful names that describe the data.

AVOID WRONG INFORMATION

BAD:

`Fruits` in a `carBrands` array?

GOOD:

Use accurate names.

MAKE SIGNIFICANT DISTINCTIONS

BAD:

`fruits` vs `fruits1`

GOOD:

Clearly differentiate variables.

USE PRONOUNCEABLE NAMES

BAD:

`x` , `y` , `a` , `b`

GOOD:

`customerName` , `orderDate`

EASY TO SEARCH

BAD:

Why does the loop run until `20`? (Magic number)

GOOD:

Use named constants.

HUNGARIAN NOTATION

BAD:

Prefixing variable names with type (`strName`)

GOOD:

Avoid unnecessary type indicators.

AVOID MENTAL MAPPING

Readers shouldn't have to translate cryptic names.

CLASS NAMES

MUST BE NOUNS.

GOOD: `Customer`, `Account`

BAD: `SaveCustomer`, `UpdateAccount`

METHOD NAMES

MUST BE VERBS.

GOOD: `postCustomer()`, `saveAccount()`

SIGNIFICANT CONTEXT

BAD:

`state` (What does it represent?)

GOOD:

`addressState`

DON’T PUT UNNECESSARY CONTEXT

If you’re in an `Address` class, don’t prefix everything with `Address`.

COMMENTS

BAD COMMENTS

Comments should not justify bad code.

Code should be self-explanatory.

TYPES OF BAD COMMENTS

Redundant

Describing what code should already express

COMMENTED-OUT CODE (the worst!)

GOOD COMMENTS

Licenses

Explanations of complex algorithms

Workarounds for external issues

FUNCTIONS

KEEP FUNCTIONS SMALL

SMALL

EVEN SMALLER (Aim for max 20 lines)

SINGLE RESPONSIBILITY PRINCIPLE

A function should do ONE THING well.

TOP-DOWN READABILITY

Organize code so reading from top to bottom makes sense.

PARAMETERS

Ideal: 0 PARAMETERS

Avoid boolean flags; they indicate multiple responsibilities.

DON’T REPEAT YOURSELF (DRY)

Avoid duplicate logic.

THANK YOU!